SERVICENOW PROJECT REPORT

IMPLEMENT CLIENT SCRIPT & UI POLICY

A Practical Guide to Client-Side Form Customization in ServiceNow

Done by: X.PUSPHA JOESHINY

NM id: AB1E570AC38612A9F314191494CBF83D

Project Type: Academic / Technical Project
Platform: ServiceNow
Topic: Client Script and UI Policy

Prepared for Project Submission

Table of Contents

1. Abstract
2. Introduction
3. Project Objectives
4. ServiceNow Overview
5. Client Script
6. UI Policy
7. Project Scenario
8. Implementation
9. Sample Client Scripts
10. UI Policy Configuration
11. Testing
12. Client Script vs UI Policy
13. Advantages and Limitations
14. Best Practices
15. Expected Output
16. Conclusion
17. Viva Questions
18. References

1. Abstract

This project demonstrates how Client Scripts and UI Policies can customize form behavior in ServiceNow. Client Scripts execute JavaScript in the user's browser and are useful for validation, dynamic values, and form interactions. UI Policies provide a configuration-based way to make fields mandatory, visible, or readonly. An Incident Management scenario demonstrates how both features can work together.

2. Introduction

ServiceNow is a cloud-based platform used for IT service management, workflow automation, and enterprise service management. Forms can be customized without modifying core application code. Client Scripts and UI Policies are important client-side configuration features that improve user experience and data quality.

3. Project Objectives

Understand Client Scripts and UI Policies; configure them on a ServiceNow form; validate user input; dynamically control field visibility, mandatory state, and read-only state; understand execution timing; and test the final form behavior.

4. ServiceNow Overview

ServiceNow stores business records in tables such as Incident, Problem, Change Request, and User. A form is the interface used to create or update a record. Client-side configurations run in the user's session and can change presentation or behavior before a record is submitted.

5. Client Script

A Client Script is a JavaScript configuration that runs on the client side of a ServiceNow form. It can access form fields through the g_form API and react to events. Common types are onLoad, onChange, onSubmit, and onCellEdit.

5.1 Client Script Types

onLoad: Runs when the form loads. **onChange:** Runs when a selected field changes. **onSubmit:** Runs when the user submits the form and can prevent submission by returning false. **onCellEdit:** Used for list editing scenarios.

6. UI Policy

A UI Policy is a declarative configuration that dynamically changes field behavior based on a condition. A UI Policy can make fields mandatory, visible, or read-only. UI Policies are preferred when the requirement is purely about form presentation and does not need custom JavaScript.

6.1 UI Policy Components

A UI Policy normally contains a table, condition, active setting, and optional Reverse if false setting. UI Policy Actions specify what happens to individual fields when the condition is true.

7. Project Scenario

In this project, an Incident form is customized. If the Incident is marked Resolved, Resolution Code and Resolution Notes become mandatory. If Category is Hardware, the Configuration Item field becomes

visible. If Impact is High, a Client Script validates that Business Justification has been provided before submission.

8. Implementation

Open the ServiceNow instance and navigate to the Incident table. Configure UI Policies for simple fieldstate changes and Client Scripts for scripted validation. Test each rule independently and then test their combined behavior.

8.1 UI Policy: Resolution Fields Mandatory

Create a UI Policy on Incident. Condition: State is Resolved. Set it Active. Add UI Policy Actions for Resolution Code and Resolution Notes, setting Mandatory to true. Save and test by changing State to Resolved.

8.2 UI Policy: Hardware Category

Create a UI Policy on Incident. Condition: Category is Hardware. Add a UI Policy Action for Configuration Item and set Visible to true. Configure the default/reverse behavior as appropriate for the form.

9. Sample Client Scripts

The following examples illustrate common ServiceNow client-side scripting patterns. Field names can differ between instances, so replace custom field names with the actual dictionary names used in your application.

9.1 Example: onChange Client Script

Purpose: React to a field value changing and update another field.

```
function onChange(control, oldValue, newValue, isLoading, isTemplate) { if (isLoading || newValue == '') { return; } if (newValue == 'hardware') { g_form.setVisible('cmdb_ci', true); } }
```

9.2 Example: onSubmit Validation

Purpose: Prevent submission when a required business condition is not satisfied.

```
function onSubmit() { if (g_form.getValue('impact') == '1' && g_form.getValue('u_business_justification') == '') { g_form.addErrorMessage( 'Business Justification is required for High Impact incidents.' ); return false; } return true; }
```

9.3 Example: onLoad Client Script

Purpose: Apply a form change when the record is loaded.

```
function onLoad() { g_form.setReadOnly('number', true); }
```

10. UI Policy Configuration

Example configuration:

UI Policy	Condition	Field	Action
Resolution Mandatory	State = Resolved	Resolution Code	Mandatory = True
Resolution Mandatory	State = Resolved	Resolution Notes	Mandatory = True

Hardware Display	Category = Hardware	Configuration Item	Visible = True

Implementation steps: Navigate to the relevant UI Policies module in your ServiceNow application. Select New, choose the table, enter the condition, activate the policy, and create UI Policy Actions. Open the form and test the condition.

11. Testing

Testing should verify both positive and negative conditions. Test with new and existing records and confirm that the intended fields change state and validation messages appear when required.

Test Case	Action	Expected Result
TC-01	Open new Incident	Form loads normally
TC-02	Set State to Resolved	Resolution Code and Notes become mandatory
TC-03	Change Category to Hardware	Configuration Item becomes visible
TC-04	Set Impact to High; leave justification empty	Submission is blocked with an error
TC-05	Enter justification and submit	Client validation permits submission

12. Client Script vs UI Policy

Feature	Client Script	UI Policy
Purpose	Scripted client-side behavior	Declarative form behavior
Coding	JavaScript	Mostly configuration
Mandatory / Visible / Read-only	Can control with g_form	Directly supported
Complex validation	Suitable	Limited
Typical events	onLoad, onChange, onSubmit, onCellEdit	Condition-driven

13. Advantages and Limitations

Client Script: flexible logic, dynamic validation, and event-based behavior. Limitations include maintenance effort and the fact that client-side logic is not a security boundary. **UI Policy:** simple configuration for mandatory, visible, and read-only states. Complex business logic may require scripting.

14. Best Practices

Use UI Policies for simple field-state requirements. Keep Client Scripts focused. Use meaningful names and descriptions. Avoid unnecessary server calls. Use field names rather than labels in scripts. Test new and existing records. Do not rely on client-side controls as the sole mechanism for security or server-side data integrity.

15. Expected Output

After implementation, the Incident form responds dynamically to user input. Resolved incidents require resolution information, Hardware incidents display the Configuration Item field, and high-impact incidents cannot be submitted without the required justification.

16. Conclusion

This project demonstrates the implementation of Client Scripts and UI Policies in ServiceNow. UI Policies provide an efficient declarative method for common form changes, while Client Scripts provide greater flexibility for event-driven logic and validation. Together, they help create interactive and maintainable ServiceNow forms.

17. Viva / Interview Questions

1. What is a Client Script?
2. What are the types of Client Scripts?
3. What is a UI Policy?
4. How does a UI Policy differ from a Client Script?
5. What is g_form?
6. When would you use onSubmit?
7. Can a Client Script provide security?
8. What is Reverse if false in a UI Policy?
9. Why should field names be used in scripts?
10. How would you test client-side configurations?

18. References

ServiceNow Product Documentation — Client Scripts and GlideForm (g_form).

ServiceNow Product Documentation — UI Policies and UI Policy Actions.

ServiceNow Developer documentation and training materials.